```
    if (status === "unsold") {
      return "bg-white hover:bg-blue-300"
    } else if (status === "held") {
      return "bg-green-500"
    } else {
      return "bg-red-500"
    }
  }

  function changeState(): void {
    clickHandler(seatNumber)
  }

  const cssClass = "p-4 m-2 border-black border-4 text-lg"

  return (
    <td>
      <span
        className={`${cssClass} ${stateDisplayClass()}`}
        onClick={changeState}>
        {seatNumber + 1}
      </span>
    </td>
  )
}

export default Seat
```

We have an interface that defines the props, including the clickHandler prop, which is defined as a function that takes a number and does not have a return value. When we declare the Seat, we use JavaScript destructuring to assign the props to values directly rather than use a props object.

We define two functions: the first converts the seat's status to a Tailwind CSS class to draw the background color. It's what we had before plus a new entry for invalid seats. On a click, we now call the second function, changeState, which itself calls our clickHandler prop. Remember, the clickHandler is the function defined by Row and passed into the seat's props. So when the seat is clicked, the OnClick is invoked, the clickHandler function is called, and control goes back to the Row, where the statuses are updated and the row redrawn.